

Silnik wyceny kosztowej dla dystrybucji B2B

Rozwiązanie, które dla każdego koszyka liczy *pełny koszt obsługi tego konkretnego klienta* — zakup, robocizną, opakowanie, magazyn i dowóz — dokłada marżę docelową, pilnuje bezpieczeństwa i zapisuje całą ścieżkę tak, że handlowiec potrafi ją przeczytać klientowi na głos.

Z CZEGO SKŁADA SIĘ CENA — PRZYPADEK REFERENCYJNY, 36,10 PLN / SZT.



20,00 cena zakupu 0,67 robocizna 0,32 opakowanie 0,77 magazyn i kapitał 14,35 marża docelowa

11	21	42	908
komponentów ceny, 9 policzonych, 2 jawnie zadeklarowane jako brakujące	encji parametrów i księgi audytowej	ekrany backendu + portal zamawiania dla klienta	testy jednostkowe w obu modułach, wszystkie przechodzą

ZAKRES

pakiet @open-mercato/pricing-engine
moduł aplikacyjny
distributor_workspace

BRANŻA REFERENCYJNA

hurt HoReCa — chemia,
opakowania, papier

TRYB PRACY

obserwacyjny (shadow):
silnik liczy, faktura bez zmian

DATA

19 września 2026

01 Problem, który to rozwiązuje

Dystrybutor sprzedaje z cennika i z negocjacji. Nikt nie wie, ile naprawdę kosztuje obsłużenie konkretnego zamówienia konkretnego klienta — więc nikt nie wie, na czym zarabia.

Koszt niewidoczny	W całej platformie nie istniała żadna kolumna z ceną zakupu — ani w katalogu, ani w magazynie, ani w sprzedaży. Marża była wyliczana po stronie sprzedaży z ceny, której nikt nie porównywał z kosztem.
Koszt obsługi	Zamówienie złożone telefonicznie na jeden karton i zamówienie z pliku na paletę kosztują dystrybutora skrajnie różne pieniądze, a wyceniane były identycznie. Robocizna, opakowanie, miejsce paletowe i dowóz nie trafiały na żadną pozycję.
Pozycja zakupowa	Roczny wolumen u dostawcy i progi rabatowe to największa pojedyncza dźwignia na cenie zakupu — i nie były reprezentowane nigdzie w systemie.
Brak audytu	Handlowiec nie potrafił uzasadnić ceny. Oferty sprzed trzech miesięcy nie dało się odtworzyć ze stawkami, które wtedy obowiązywały.
Skalowanie	Wdrożenie u drugiego dystrybutora oznaczało forka kodu, bo każda stawka była zaszyta w implementacji.

ODPOWIEDŹ

Reguły są kodem, **wszystkie stawki są danymi** — wersjonowanymi w czasie i edytowalnymi z panelu. Każda wycena zapisuje się razem ze swoimi wejściami, wersją zestawu parametrów i zdaniem wyjaśniającym na każdy składnik. Drugi dystrybutor to jeden pakiet danych, nie nowa gałąź kodu.

02 Jak to działa — zasada w sześciu punktach

1. Reguły w kodzie, parametry w bazie

Każdy składnik ceny to funkcja TypeScript o ustalonej pozycji w potoku. Każda stawka, próg i procent to wiersz w bazie z zakresem obowiązywania i datami ważności. Nie ma DSL-a ani „generycznego silnika reguł”, którego nikt nie umie zdebugować — jest jedenaście nazwanych kroków, które zawsze wykonują się w tej samej kolejności.

2. Parametr jest wersjonowany w czasie i ma zakres

Ta sama stawka może być inna dla klienta, dla grupy klientów, dla produktu, dla grupy produktowej i globalnie. Wygrywa najbardziej szczegółowy zakres, a w jego obrębie — najnowszy wiersz obowiązujący na dzień wyceny.

precedencja	klient → grupa klientów → produkt → grupa produktowa → globalnie
ważność	valid_from ≤ data_wyceny AND (valid_to pusty OR valid_to > data_wyceny)
remis	wygrywa najnowszy valid_from — korekta zastępuje wiersz bez domykania starego

Cena paliwa jest szeregiem czasowym, nie pojedynczą liczbą: wycena z marca odtwarza notowanie, które faktycznie obowiązywało w marcu.

3. Wycena jest deterministyczna i odtwarzalna

Ten sam koszyk z tą samą datą daje tę samą cenę co do grosza. Na każdej wycenie stemplowana jest wersja zestawu parametrów, więc ofertę można odtworzyć dokładnie, także po tym, jak stawki się zmieniły.

4. Arytmetyka pieniędzy nie ma prawa dryfować

Cała matematyka idzie przez jedną warstwę na liczbach całkowitych BigInt skalowanych o 10^{12} — zero liczb zmiennoprzecinkowych. Po każdym składniku wynik jest kwantyzowany do czterech miejsc, tak żeby *drukowany wodospad sumował się do drukowanej ceny*, a $\text{cena} \times \text{ilość}$ równała się wartości pozycji. Oba warunki są przypięte testami.

5. Zero wymyślonych danych

Kiedy silnik nie zna liczby, nie zgaduje jej. Zwraca zero z nazwanym ostrzeżeniem i obniża deklarowaną wiarygodność składnika. Brak jednostki przy wymiarach produktu znaczy „nieznana”, nigdy „pewnie centymetry”.

6. Jedno zapytanie na koszyk, nie na pozycję

Wszystko, co składniki czytają, jest pobierane *przed* pętlą: 11 zapytań o parametry (równolegle), snapshot katalogu i maksymalnie 4 zapytania do magazynu. Koszyk o pięćdziesięciu pozycjach kosztuje tyle samo odczytów co jednopozycyjny — dzięki temu doradca może przeliczyć kilkanaście hipotetycznych wariantów koszyka bez ani jednego dodatkowego odczytu z bazy.

03 Potok wyceny — jedenaście kroków

Cena nie jest jedną formułą. Jest narastającą sumą nazwanych składników, w stałej kolejności, z których każdy podaje swoją wartość, swoje wejścia, swoją wiarygodność i jedno zdanie wyjaśnienia.

POTOK: 9 SKŁADNIKÓW POLICZONYCH, 2 ZADEKLAROWANE JAKO BRAKUJĄCE

#	SKŁADNIK	TYP	CO LICZY I Z CZEGO	WIARYGODNOŚĆ
1	product_cost	dodaje	Cena zakupu z ostatniej dostawy pomniejszona o obowiązujący próg rabatowy u dostawcy. Nigdy średnia okresowa — średnia ukrywa moment, w którym dostawca podniósł cenę. <i>Wyjątek</i> : pozycja przeliczanego dokumentu może przynieść własny koszt zakupu i wtedy jest on brany dosłownie (patrz niżej).	MEASURED do 60 dni od dostawy
2	operational_cost_base	dodaje	Robocizna obsługi zamówienia. Suma kroków procesu (przyjęcie zamówienia, kompletacja, pakowanie, wydanie, fakturowanie, windykacja) przeliczonych po stawce roli z narzutem. Kroki „na zamówienie” rozkładają się na pozycje, kroki „na pozycję” zostają przy niej.	MEASURED / DEFAULT
3	packaging_cost	dodaje	Materiał i czas pakowania — osobno dla sztuki, kartonu i palety. Liczba opakowań z przeliczników jednostek katalogu, zawsze w dół (1,99 kartonu to jeden karton plus luz).	MEASURED / DEFAULT
4	warehouse_cost	dodaje	Zajmowane miejsce + zamrożony kapitał. Udział w miejscu paletowym liczony jako większa z dwóch: objętościowa i wagowa. Kapitał finansuje towar, więc naliczany jest od ceny zakupu, nie od narostego kosztu obsługi.	ESTIMATED — sufit
5	logistics_cost	dodaje	Realny koszt kursu: paliwo na trasę w obie strony, czas kierowcy z narzutem, udział w koszcie stałym auta — podzielony przez typową liczbę przystanków i rozłożony na pozycje koszyka.	ESTIMATED — sufit
6	product_aspects	mnoży	Korekta za fizyczność towaru — ciężki ($> 25 \text{ kg} \rightarrow \times 1,04$), gabarytowy ($> 0,25 \text{ m}^3 \rightarrow \times 1,06$), współczynnik grupy produktowej. Mnożniki są twarde ograniczone do przedziału (0; 5]; wartość spoza pasma jest odrzucana z ostrzeżeniem, nie stosowana.	MEASURED / DEFAULT

#	SKŁADNIK	TYP	CO LICZY I Z CZEGO	WIARYGODNOŚĆ
7	customer_profile	—	Niezaimplementowany — świadomie. Miał mnożyć cenę iloczynem dziewięciu wskaźników zachowania klienta. Tenant nie ma jeszcze ani jednego aktywnego wskaźnika, więc komponent zwracałby dla każdego klienta dokładnie 1,0000 — a to gorsze niż jego brak, bo <i>wyglądałby</i> na działający.	BRAK DANYCH
8	volume_effect	—	Niezaimplementowany — zbędny. Miał być progowym mnożnikiem za wolumen. Koszty stałe na zamówienie amortyzują się jak $1/q$, więc cena jednostkowa spada z ilością sama z siebie: na danych referencyjnych 1 szt. wycenia się na 107,61, a 50 szt. na 68,04 bez żadnego mnożnika. Dołożenie go rabatowałoby ten sam wolumen dwa razy.	ZASTĄPIONY
9	target_margin	mnoży	Narzut docelowy na całość kosztu, z zakresem (inny dla grupy produktowej, inny dla klienta). Raportowana obok marża jest wyprowadzana, nigdy konfigurowana.	MEASURED / DEFAULT
10	guardrails	mnoży	Bezpieczniki: cena minimalna, marża minimalna, limit rabatu, cena negocjowana, drabinka terminu przydatności i podłoga zapasu martwego. Szerzej w rozdziale 07.	MEASURED
11	rounding	dodaje	Zaokrąglenie do siatki polityki (np. grosza) z opcjonalną końcówką psychologiczną. Nigdy nie schodzi poniżej obowiązującego progu — jeśli zwykłe zaokrąglenie przebiłoby podłogę, bierze najmniejszą wartość na siatce, która jeszcze ją trzyma.	MEASURED

DLACZEGO BRAKUJĄCE SKŁADNIKI SĄ WYPISANE, A NIE POMINIĘTE

Ekran pokrycia danych listuje wszystkie jedenaście pozycji z powodem nieobecności dwóch z nich. Składnik, którego nie ma na liście, jest niewidoczny; składnik z powodem jest decyzją, którą można zakwestionować.

Dwa źródła kosztu zakupu — i dlaczego oba są potrzebne

Wycena nowej oferty i ponowna wycena dokumentu sprzed pół roku to dwa różne pytania, więc czytają koszt towaru z dwóch różnych miejsc:

ŹRÓDŁO	KIEDY I DLACZEGO
Ostatnia dostawa pricing_purchase_positions	Ścieżka domyślna — kalkulator, doradca, portal, każda nowa oferta. Odpowiada na pytanie „ <i>ile kosztowałoby mnie sprzedanie tego dzisiaj</i> ”: cena z ostatniej dostawy pomniejszona o próg rabatowy, ze znacznikiem świeżości i ostrzeżeniem po 60 dniach.
Sam dokument purchaseUnitCostNet na pozycji	Gdy pozycja przynosi koszt zafakturowany dla niej w systemie źródłowym. Wtedy ta liczba jest brana dosłownie: bez progu rabatowego, bez ostrzeżenia o wieku, wiarygodność MEASURED — bo to jest kwota <i>zapłacona</i> , nie oszacowanie. Nie wymaga istnienia pozycji zakupowej w silniku.

DLACZEGO TO ROZRÓŻNIENIE JEST KONIECZNE

Ponowna wycena historycznego WZ musi pokazywać ten sam koszt towaru, który pokazuje system źródłowy. Nowsza dostawa jest *kosztem odtworzenia*, a zestawienie jednego z drugim raportowałoby różnicę metody wyceny tak, jakby była różnicą zysku. Reguła właściciela brzmi wprost: ten sam produkt, ten sam koszt towaru po obu stronach porównania.

Dziś jedynym miejscem korzystającym z drugiej ścieżki jest ekran porównania cen (rozdział 11). Wszystkie pozostałe wyceny idą ścieżką ostatniej dostawy.

Narzut a marża — rozróżnienie, które rozjeżdża rozmowy

Operator konfiguruje **narzut na koscie**, a myśli o **marży na cenie**. System przechowuje jedno i raportuje drugie, nigdy ich nie myląc:

cena = koszt × (1 + narzut)
66% narzutu = 39,76% marży

marża = (cena - koszt) / cena
(podłogi bezpieczeństwa są zawsze wyrażone jako marża)

04 Jak wygląda jedna wycena

Poniżej rzeczywisty, przypięty testem przypadek referencyjny: 24 sztuki produktu o koszcie zakupu 20,00 PLN, zamówienie złożone plikiem strukturalnym.

product_cost		+20,0000
operational_cost_base		+0,6672
packaging_cost		+0,3152
warehouse_cost		+0,7654
logistics_cost		0,0000
product_aspects		×1,0000
target_margin		×1,6600
guardrails		×1,0000
rounding		-0,0013
Cena jednostkowa netto		36,1000

■ koszt własny ■ marża docelowa ■ zero z ostrzeżeniem – brak danych ■ bez zmiany

Wynik pozycji

	NA EKRANIE	W KSIĘDZE
Koszt własny / szt.	21,75	21,7478
Cena / szt.	36,10	36,1000
Wartość pozycji	866,40	866,4000
Narzut	65,99 %	65,9938 %
Marża	39,76 %	39,7568 %

Cała arytmetyka idzie na czterech miejscach; do dwóch zaokrągla dopiero ostatni krok — ten, który czyta człowiek. Dzięki temu powtarzane działania nie dryfują, a księga audytowa zachowuje pełną precyzję.

Ta sama pozycja, inny kanał

Jedyne, co się zmienia, to mnożnik czasu na kroku „przyjęcie zamówienia”. Skala jest zmierzona, nie wymyślona — i jest to najczytelniejszy argument handlowy, jaki ten system produkuje:

KANAŁ ZAMÓWIENIA	MNOŻNIK	KOSZT PRZYJĘCIA ZAMÓWIENIA	UWAGA
Zamówienie powtarzalne (z szablonu)	0,25	2,59 PLN	zachowanie, nie kanał — dodatkowo -10% na kompletacji

Co silnik sam o sobie mówi

Ta sama wycena zwraca trzy ostrzeżenia, bez których liczba byłaby ładniejsza, niż na to zasługuje:

brak przeliczników opakowań

rotacja magazynowa założona

brak strefy dostawy

Trzecie z nich jest powodem, dla którego `logistics_cost` wynosi zero: kontekst nie niósł strefy, więc silnik *odmówił wymyślenia odległości*, zamiast podstawić średnią.

KANAŁ ZAMÓWIENIA	MNOŻNIK	KOSZT PRZYJĘCIA ZAMÓWIENIA	UWAGA
Plik strukturalny	0,35	3,63 PLN	docelowy sposób współpracy
Plik niestandardowy	0,90	9,33 PLN	ktoś musi go przepisać
E-mail	1,30	13,48 PLN	
SMS	2,30	23,85 PLN	
Telefon	2,47	25,61 PLN	siedmiokrotność pliku
Wizyta przedstawiciela	4,00	41,48 PLN	

DLACZEGO TO JEST WAŻNE

Klient, który przejdzie z telefonu na plik, obniża **koszt**, a nie dostaje rabatu. Oszczędność da się pokazać w złotych, obronić przed zarządem i oddać klientowi bez naruszania procentowej marży. To jedyna dźwignia cenowa w całym systemie, której nie trzeba nikomu tłumaczyć dwa razy.

05 Koszty wspólne — kto płaci za kurs

Dowóz i obsługa zamówienia nie należą do żadnej pozycji z osobna. Rozkładają się proporcjonalnie do wartości kosztowej pozycji, policzonej *przed* uruchomieniem potoku.

```
udział_pozycji = (koszt_zakupu × ilość) / Σ(koszt_zakupu × ilość)
koszt_dowozu_na_pozycję = koszt_kursu / typowa_liczba_przystanków × udział_pozycji
przy zerowym koszcie bazowym: podział równy między pozycje
```

Podstawą udziału jest **wstępny szacunek kosztu, nie wynik potoku** — inaczej alokacja zależałaby od składników, które sama zasila, i wycena przestałaby być rozstrzygalna. Dwie pozycje po 10 i 30 sztuk tego samego towaru dostają udziały 25% i 75%.

Konsekwencja architektoniczna: udziały zależą od *całego* koszyka, więc żadnej pojedynczej pozycji nie da się przeliczyć w oderwaniu. Każda symulacja — także każda z kilkunastu hipotez doradcy — przelicza koszyk w całości. Dlatego cały odczyt z bazy dzieje się raz, przed pętlą.

06 Uczciwość danych — trzy poziomy wiarygodności

Każdy składnik deklaruje, na czym stoi. To nie jest ozdoba interfejsu — to jedyny mechanizm, który powstrzymuje system przed prezentowaniem założeń jako pomiarów.

POZIOM	ZNACZENIE	PRZYKŁAD Z POTOKU
MEASURED	Liczba pochodzi z faktu zapisanego w systemie	cena z ostatniej dostawy sprzed 12 dni
ESTIMATED	Liczba wyprowadzona z danych, ale przez założony model	udział w miejscu paletowym liczony z kopertą palety EUR
DEFAULT	Silnik nie ustalił faktu i użył stałej z kodu	brak wiersza kosztu magazynu → stawka założona

Składanie zawsze wybiera **ślabsze ogniwo**. Dwa składniki mają twardy sufit **ESTIMATED** i nie mogą go przekroczyć nigdy: koszt magazynu (bo geometria miejsca paletowego jest założeniem, nie inwentaryzacją regałów) i koszt logistyki (bo dystans i czas jazdy w strefie to średnie z przeszłych kursów).

Sama obecność wiersza parametru niczego nie dowodzi. Jeśli wiersz niesie trzy z czterech potrzebnych liczb, brakująca po cichu spadnie na stałą z kodu — więc składnik sprawdza *komplet* i przy niepełnym payloadzie raportuje **DEFAULT**, choć formalnie „jest skonfigurowany”.

Rejestr pokrycia danych

Osobny ekran wypisuje wszystkie jedenaście składników z ich źródłem, świeżością, deklarowaną wiarygodnością i powodem braku. Rejestr jest budowany **z definicji potoku, nie z tego, jakie wiersze akurat istnieją** — składnik bez wpisu pokrycia pokazuje się jako brakujący, zamiast zniknąć z listy.

Rotacja magazynowa — co jest naprawdę mierzone

Koszt magazynu chce wiedzieć, ile dni towar leży. Może to policzyć z ruchów magazynowych:

```
okno 180 dni; wymagane min. 30 dni obserwacji
dienne_wydanie = wydane_w_oknie / dni_od_najstarszego_ruchu_w_oknie
dni_pokrycia    = min(stan / dienne_wydanie, 365)
```

Dni obserwacji liczą się **od najstarszego ruchu**, a nie od szerokości okna — towar przyjęty 60 dni temu ma 60 dni historii, a dzielenie przez 180 dwukrotnie zaniżyłoby popyt i potroiło pokrycie. Ruch wewnątrz magazynu (przestawienie palety) liczy się jako zero, bo netto niczego nie wydaje.

OGRANICZENIE, KTÓRE TRZEBA ZNAĆ

Żadna ścieżka produkcyjna nie zapisuje jeszcze ruchów typu *pick / ship*. Poza tenantem demonstracyjnym rotacja pozostaje na wartości domyślnej (30 dni) — i mówi o tym wprost nazwanym ostrzeżeniem, zamiast udawać pomiar.

07 Bezpieczniki — czego cena nie może zrobić

Bezpieczniki to jeden składnik potoku, ale pięć różnych reguł o ustalonym pierwszeństwie. Wszystkie są wyrażone jako marża na cenie, wszystkie mają zakres i daty ważności.

REGUŁA	DZIAŁANIE
Cena negocjowana	Cena uzgodniona indywidualnie z klientem, z datą wygaśnięcia. Po terminie przestaje obowiązywać automatycznie — bez tego rabat sprzed dwóch lat żyje wiecznie.
Limit rabatu	Ogranicza <i>wyłącznie</i> cenę negocjowaną — jedyne miejsce, gdzie cenę ustala człowiek, a nie silnik. Cena wyliczona przez silnik nie jest ograniczana, bo już zawiera koszt.
Cena minimalna	Twarda podłoga kwotowa, niezależna od kosztu.
Marża minimalna	Podnosi cenę do $\text{koszt} / (1 - \text{marża_min})$. Przy domyślnych 8% to $\text{koszt} / 0,92$.
Drabinka terminowa i podłoga zapasu martwego	Otwierają cenę <i>poniżej</i> zwykłych podłóg. To jedyne dwa przypadki, w których marża minimalna nie obowiązuje — i jest to cały sens tego pozwolenia.

Drabinka terminu przydatności

Towar z kończącym się terminem ma inną ekonomię niż towar zdrowy: koszt zakupu jest już zatopiony, a alternatywą dla niskiej ceny nie jest cena wysoka, tylko **spisanie w straty** — przy chemii dodatkowo płatne, bo utylizacja kosztuje.

POZOSTAŁY OKRES ŻYCIA PODŁOGA CENY

powyżej 25%	normalna marża minimalna
25% – 10%	marża spada do 5%: koszt / 0,95
10% – 5%	podłoga = koszt obsługi (sprzedaż po kosztach)
poniżej 5%	podłoga ujemna: – waga_kg × stawka_utylicacji — oddanie za złotówkę bije zapłaceniu za zniszczenie

NAJWAŻNIEJSZA DECYZJA PROJEKTOWA W CAŁYM MODULE

Drabinka **obniża podłogę, nigdy nie ustawia ceny**. Gdyby ustawiała, każdy klient pytający o produkt, którego jedna paleta się starzeje, dostałby ofertę po kosztach — łącznie z klientem, który zapłaciłby pełną stawkę, i dla całej części zamówienia, która nigdy nie była zagrożona. Wygasający termin kupuje *pozwolenie* zejścia nisko; decyzję podejmuje człowiek, widząc dokładnie, ile ma zapasu ceny i jaki odpis w złotych jest alternatywą.

Przy zamówieniu większym niż zagrożona partia podłoga jest **ważona po partiach, które FEFO faktycznie zużyje** — nie ma rabatu na sztuki, które pójda ze świeżej dostawy. Kolejność wydań jest deterministyczna (najwcześniejszy termin, remisy po numerze partii), bo cena zmieniana przez niestabilne sortowanie byłaby nieaudytowalna.

Zaokrąglenie nie ma prawa przebić podłogi

Bezpieczniki publikują obowiązującą podłogę nawet wtedy, gdy nie przesunęły ceny. Zaokrąglenie ją czyta i jeśli zwykle zaokrąglenie w dół zesłoby o grosz poniżej progu, bierze najmniejszą wartość na siatce, która próg jeszcze trzyma.

08 Zapas martwy — pieniądze, które stoją na regale

Osobny ekran odpowiada na pytanie, którego cennik nie zadaje: co leży, od jak dawna, ile to kosztuje miesięcznie i za ile warto to sprzedać dzisiaj.

Klasyfikacja — pięć stanów, nie dwa

KLASA	WARUNEK	CO TO ZNACZY
never_sold	zero sprzedaży w całej historii	Zakup błędny w dniu, w którym go zrobiono — odrębna klasa, bo to inna choroba niż wygaszony popyt i jedyna bez jakichkolwiek danych o tym, jak długo potrwa wyprzedaż
dead	180 dni bez sprzedaży	Poprawny zakup, którego popyt się skończył
dying	90 dni bez sprzedaży <i>lub</i> spadek do < 25% poprzedniego okresu	Trend się załamuje — jeszcze jest czas zareagować
slow	pokrycie zapasu > 120 dni	Nie dostaje przeceny. Wolnorotujący towar <i>się sprzedaje</i> , po prostu kupiono go za dużo. Przecena niszczy marżę na popycie, który i tak nadejdzie — to sygnał zakupowy, nie cenowy
healthy	pozostałe	

Trzy supresory — czyli kiedy oskarżenie zostaje wstrzymane

Werdykt ryzykowny może zostać wstrzymany, ale wiersz nie znika: zachowuje metryki i pokazuje, co zostało wstrzymane i dlaczego.

- **Produkt nowy** — mniej niż 90 dni dostępności. Liczona jest data *pierwszego* przyjęcia, nigdy ostatniego: przy liczeniu od ostatniego produkt uzupełniany co cztery tygodnie raportował wiecznie cztery tygodnie dostępności i był

wstrzymywany bez końca (zmierzone: 85 wstrzymanych, 0 oskarżonych).

- **Sezonowość** — wymaga *pięciu* warunków naraz: co najmniej rok historii, najwyżej 5 miesięcy szczytowych, co najmniej 8 zamówień w historii, nadchodzący szczyt w ciągu 6 miesięcy i bieżący miesiąc poza szczytem. Bez progów 5/8 zbiór referencyjny dawał 26 wstrzymań przy 9 faktycznie sezonowych produktach.
- **Decyzja operatora** — ktoś to już przejrzał i odłożył do wskazanej daty.

ODWROTNA REGUŁA, ŚWIADOMA

Gdy okres dostępności jest *nieznany*, werdykt **przechodzi** z nazwanym brakiem. Wstrzymanie ukrywałoby produkt dokładnie dlatego, że system wie o nim najmniej — zmierzone: ukrywało to wszystkie 15 pozycji nigdy niesprzedanych.

Koszt utrzymania i rekomendowana podłoga

Ta sama arytmetyka co składnik magazynowy w cenie — dystrybutor, który zobaczy jedną liczbę w ofercie i inną na tym ekranie, nie uwierzy żadnej.

```
na_sztukę_miesięcznie = udział_w_miejscu × stawka_miejsca + koszt_zakupu × stopa_kapitału/12
pozycja_miesięcznie   = na_sztukę_miesięcznie × stan
zamrożony_kapitał      = stan × koszt_zakupu
```

```
podłoga (dead / never_sold) = max(0, koszt_zakupu - na_sztukę_miesięcznie × horyzont)
horyzont = 6 miesięcy (dead) · 12 miesięcy (never_sold)
podłoga (dying) = koszt_obsługi / (1 - 5%)
```

Zejsście poniżej ceny zakupu jest uzasadnione dokładnie jednym argumentem: **koszt zakupu jest zatopiony, a utrzymanie nie jest**. Czynsz i zamrożony kapitał naliczają się tak długo, jak paleta stoi. Sprzedaż dziś za koszt – utrzymanie przez horyzont wyprzedaży zostawia dystrybutora dokładnie tam, gdzie zostawiłoby trzymanie przez ten horyzont i sprzedaż po koszcie — i ściśle lepiej niż dużo bardziej prawdopodobne trzymanie i spisanie. *Przyszłe utrzymanie jest całą licencją na zejście poniżej kosztu, dlatego podłoga jest ograniczona nim i niczym innym.*

PODŁOGA JEST AUTORYZOWANA, NIE WYWNIOSKOWANA

Silnik **nigdy** nie wyprowadza podłogi zapasu martwego w trakcie wyceny. Czyta wstecz to, co człowiek zatwierdził, ze snapshotu zapisanego przy decyzji — liczby w nim to liczby, które były wtedy na ekranie, nie to, co to samo zapytanie zwróciłoby dziś. Snapshot liczy się po stronie serwera i nigdy nie jest czytany z zapytania, bo klient mogący przysłać własne liczby autoryzowałby dowolną cenę. Decyzje są nieusuwalne (tylko dopisywanie), a autoryzacja wygasa w dniu przeglądu — *pozwolenie na sprzedaż poniżej kosztu, które przeżyje sytuację, jest stałym rabatem, którego nikt nie pamięta przyznać.*

Dwie drabinki — terminowa i zapasu martwego — **nie sumują się**. Wygrywa niższa podłoga. To są pozwolenia, a pozwolenia się nie kumulują: złożenie ich zesłoby poniżej kosztu mniej więcej dwa razy szybciej, niż zamierzał którykolwiek z mechanizmów.

Zmierzone na zbiorze referencyjnym

83 795 PLN zamrożonego kapitału w zapasie ryzykownym	2 703 PLN miesięcznie kosztu samego trzymania	68 477 PLN możliwe do odzyskania przy rekomendowanej podłodze	35 / 13 pozycji z werdyktem / werdyktów wstrzymanych przez supresory
---	--	---	--

Liczba zbiegła się z 82 972 PLN oszacowanymi niezależnie surowym zapytaniem SQL, zanim ten kod powstał.

09 Doradca — jak obniżyć cenę klientowi i zarobić więcej

Pięć generatorów szuka zmian w koszyku, które realnie obniżają cenę jednostkową klienta, nie naruszając zysku dystrybutora. Każda propozycja jest **przeliczona pełnym potokiem**, a nie oszacowana — czyli jest legalna cenowo z konstrukcji.

GENERATOR	SKĄD BIERZE SIĘ OSZCZĘDNOŚĆ
Próg wolumenowy volume_threshold	Zwiększenie ilości o 25 / 50 / 100%. Koszty stałe na zamówienie amortyzują się jak $1/q$, więc cena jednostkowa spada bez żadnego rabatu. Jeśli dodana ilość domyka <i>roczny</i> próg rabatowy u dostawcy dystrybutora, propozycja to sygnalizuje — ale nigdy nie twierdzi, że jedno zamówienie zarabia rabat.
Pełne opakowanie full_pack_rounding	Zaokrąglenie do pełnego kartonu lub palety. Uczciwość mechanizmu: przekroczenie granicy opakowania <i>nie jest automatycznie tańsze</i> — nalicza się całe dodatkowe opakowanie. Generator raportuje wynik przeliczenia łącznie ze wzrostem i oznacza go osobno.
Zmiana kanału order_channel_change	Najczystsza dźwignia w systemie: zmierzona oszczędność pracy na przyjęciu zamówienia, od $\times 4,00$ (wizyta) do $\times 0,35$ (plik).
Łączenie zamówień basket_consolidation	Dwa koszyki wyceniane osobno kontra jeden scalony: kroki płacone raz na zamówienie i jeden przystanek na trasie zamiast dwóch — od ok. 36 PLN w strefie miejskiej do ponad 800 PLN w strefie dalekiej.
Tańszy zamiennik cheaper_equivalent	Produkt z tej samej grupy o niższym koszcie efektywnym. Wiarygodność wymuszona na najniższą , bo grupa produktowa to kategoria, nie relacja zamienności — każda taka propozycja wymaga potwierdzenia człowieka.

Każda propozycja pokazuje obie strony bilansu

cena przed / po	co płaci klient za sztukę
zysk przed / po	ile zostaje dystrybutorowi — w złotych i w procentach, na pozycji i na całym koszyku
cena neutralna	cena, przy której klient dostaje <i>całą</i> oszczędność kosztu, a absolutny zysk dystrybutora nie zmienia się ani o grosz
podłoga	najniższa cena, jakiej silnik dotrzyma — ta sama funkcja, której używa bezpiecznik
wiarygodność	najstabszy składnik w rozbiciu wariantu

```
cena_neutralna_zyskowo = koszt_po + (cena_przed - koszt_przed)
rodzina_pośrednia      cena(φ) = cena_przed - φ × (koszt_przed - koszt_po) × (1 + narzut)
    φ = 0 → cena trzymana (cała oszczędność u dystrybutora)
    φ = 1 → narzut trzymany (oszczędność dzielona proporcjonalnie)
    φ = 1/(1+narzut) → zysk w złotych trzymany, cała oszczędność u klienta
```

Przykład różnicy: przy 66% narzutu spadek kosztu ze 100 na 90 daje cenę 149,40 i zysk 59,40 przy trzymaniu narzutu, wobec 156,00 i zysku 66,00 przy neutralności zysku. To nie jest niuans — to dwie różne polityki handlowe i system pokazuje obie.

Cele i wagi — dystrybutor mówi, co jest ważne

Operator definiuje własne cele i sam ustawia ich wagi. Metryka musi być jedną z dziewięciu, które wycena i tak już emituje — waga ma sens tylko wtedy, gdy silnik potrafi zmierzyć ważoną wielkość na konkretnej parze przed/po.

marginPercent

profitNet

revenueNet

unitCostNet

productCost

operationalCost

packagingCost

warehouseCost

logisticsCost

```
znormalizowana_zmiana = (kierunek * (po - przed)) / |przed|      [baza zerowa → ±1, nigdy 0]
wynik = Σ(waga * znormalizowana_zmiana) / Σ(waga)
```

Dzielenie przez wartość „przed” sprowadza punkt procentowy marży i złotówkę zysku na tę samą skalę. Panel pokazuje **wkład każdego celu osobno** — ranking, którego nikt nie umie rozebrać, to ranking, któremu nikt nie zaufa. Brak skonfigurowanych celów zostawia kolejność nietkniętą; wszystkie wagi na zero to legalna konfiguracja, nie błąd.

ŚWIADOMY BRAK FILTRA „BEZPIECZNIKI BIJĄ WAGI”

Taki filtr byłby nie tylko zbędny, ale **czynnie szkodliwy**. Każda propozycja przeszła już przez składnik bezpieczników, więc jest legalna. Filtr wyprowadzałby podłogę ponownie — z samej marży minimalnej — i skasowałby dokładnie te propozycje na towarze z kończącym się terminem, których operator potrzebuje najbardziej.

Deduplikacja — licznik musi zgadzać się z kartami

Dwa generatory potrafią zaproponować tę samą akcję pod dwoma tytułami. Deduplikacja działa po **tożsamości zmiany** (produkt + ilość docelowa + produkt docelowy + kanat), nigdy po rodzaju; przy kolizji wygrywa ten, który tłumaczy akcję lepiej — „dopełnij karton” bije „zamów więcej”. Propozycje bez rozróżnialnej zmiany (łączenie koszyków) są z deduplikacji wyłączone, bo zwinięcie po ich payloadzie skasowałoby cztery z pięciu wariantów scalenia: *deduplikacja nie może stać się usuwaniem*.

Wrażliwość na wolumen i podłoga rabatowa

Dwa dodatkowe narzędzia dla handlowca w tej samej odpowiedzi:

- **Krzywa ilości** — ten sam produkt wyceniony na drabinie 1 / 2 / 6 / 12 / 24 / 48 / 96 / 240 sztuk, wzbogaconej o granice opakowań i aktualną ilość z koszyka. Każdy szczebel wyceniany jako *własny jednopozycyjny koszyk*, bo inaczej ilości pozostałych pozycji wplotłyby się w odpowiedź i przestałyby to być właściwość produktu.
- **Najniższa możliwa cena** — podłoga dla marży minimalnej i zapas rabatu w procentach, liczone tą samą funkcją, którą klamruje bezpiecznik. Handlowiec może też zapytać hipotetycznie („a gdybym upierał się przy 15%?”) bez zmieniania reguły.
- **Pozycja zakupowa** — ile sztuk brakuje do następnego rocznego progu u dostawcy i ile złotych rocznie zdjęłoby to z kosztu. Informacja wyłącznie dla dystrybutora, nigdy nie trafia na powierzchnię klienta.

10 Prognoza zamówień — co ten klient kupi i kiedy

Drugi filar rozwiązania: z historii sprzedaży wyprowadzany jest rytm zamawiania klienta i przewidywany koszyk na najbliższe dostawy. Liczone na żywo przy każdym otwarciu — nigdy składowane, więc nie ma czego odświeżać ani co się nie zdezaktualizuje.

Wykrywanie rytmu

```
obserwacje agregowane do kubełków dobowych (dwie faktury tego samego dnia = jedno zdarzenie)
mediana_odstępu      = median(odstępy między dniami zakupu)
rozrzut              = MAD(odstępy)      [mediana odchyłeń, nie odchylenie standardowe]
regularyność         = 1 - (rozrzut / mediana) / 0,6
rytm_tygodniowy      = gdy ≥60% zakupów w jednym dniu tygodnia i mediana blisko wielokrotności 7
```

MAD zamiast odchylenia standardowego jest wyborem, nie stylem: jedno świąteczne podwójne zamówienie nie ma prawa zabić rozpoznanego rytmu.

Cztery bramki, które odrzucają przed policzeniem

POWÓD ODRZUCENIA	PRÓG I UZASADNIENIE
za mało zamówień	mniej niż 3 wystąpienia — produkt kupiony dwa razy to nie wzorzec
za krótka historia	rozpiętość poniżej 21 dni — trzy dostawy w jednym tygodniu to jedno uzupełnienie rozbite na faktury
brak rytmu	nie da się wyznaczyć mediany odstępu
rytm porzucony	spóźnienie większe niż $\min(21 \text{ dni}, \text{mediana odstępu})$

Każde odrzucenie wraca na ekran z powodem, liczbą wystąpień i datą ostatniego zakupu. Sekcja „Pominięto” jest częścią odpowiedzi, nie jej brakiem.

Pewność — cztery czynniki, średnia geometryczna

wsparcie	ile razy to się wydarzyło (3 zamówienia $\rightarrow$ 0,17; 8 $\rightarrow$ 1,00)
regularność	jak równe są odstępy
pokrycie	ile z oczekiwanych cykli faktycznie wystąpiło — jedyna bramka łącząca „osiem tygodniowych zamówień, potem trzy miesiące ciszy”
świeżość	jak daleko za terminem jesteśmy, z podłogą 0,15

```
pewność_surowa = exp(  $\sum \text{waga} \times \ln(\text{czynn timer})$  /  $\sum \text{waga}$  )  
wagi: wsparcie 1,0 · regularność 1,2 · pokrycie 1,0 · świeżość 0,8
```

Geometryczna, nie arytmetyczna: czynnik bliski zeru **unieważnia** predykcję, a średnia arytmetyczna pozwoliłaby trzem mocnym czynnikom przenieść jeden zabójczy. Podłoga 0,15 przy świeżości jest konieczna właśnie dlatego — bez niej wiersz dokładnie na granicy raportowałby „za niska pewność” zamiast „rytm porzucony”. Sufit 0,97 jest celowy: wiersz „100%” zachęca operatora, żeby przestał sprawdzać.

Data następnego zamówienia

```
projekcja = ostatni_zakup + round(mediana_odstępu) dni  
korekta = dociągnięcie do dnia tygodnia, w którym klient faktycznie zamawia (maks.  $\pm 3$  dni)
```

Interwał stawia datę, dzień tygodnia ją koryguje — **nigdy odwrotnie**, i nigdy dalej niż o pół tygodnia, żeby korekta nie zamieniła jednego cyklu w następny. Kalendarz dni tygodnia klienta włącza się dopiero od 12 różnych dni zamówieniowych: brak soboty wśród sześciu zamówień to przypadek, wśród dwunastu to dzień, w którym klient nie przyjmuje dostaw.

DATA SPÓŹNIONA NIE JEST ROLOWANA W PRZYSZŁOŚĆ

Klient 41 dni po terminie na produkcie miesięcznym dostałby datę za dwa tygodnie — czyli sygnał „wyluzuj” dokładnie na koncie, do którego trzeba dzwonić. Data zostaje w przeszłości, więc sortowanie po „ile dni do” wypycha najbardziej spóźnionych na samą górę listy.

Przewidywany koszyk — jednostką jest dostawa, produkt jest dowodem

Predykcje produktowe są rzutowane na **kotwice dostaw** wyprowadzone z rytmu całego klienta (do 8 cykli w przód, każda skorygowana o jego dzień tygodnia). Pozycja tygodniowa trafia do każdego koszyka, chemia dwutygodniowa — do co

drugiego. Pewność koszyka to **mediana** pewności jego linii: nie maksimum, bo jeden pewniak nie może ręczyć za sześć zgadywanek. Pewność linii wygasa z odległością ($\times 0,88$ na cykl) — piąta dostawa w przód nie ma prawa brzmieć jak jutrzejszy bus. Suma wartości pokazuje się tylko wtedy, gdy *każda* linia ma cenę i wszystkie są w jednej walucie.

Backtest — system ocenia sam siebie

Zamiast rejestru predykcji (stronniczego, bo wiersze istnieją tylko dla klientów, których ktoś otworzył) stosowany jest **replay**: historia jest cofana do 14 punktów odcięcia co dwa tygodnie, prognoza liczona na danych sprzed odcięcia i porównana z tym, co faktycznie przyszło.

```
trafienie = zakup tego samego produktu w oknie  $\pm 3$  dni od przewidzianej daty
trafność = trafienia / (trafienia + pudła)           ile z obiecanego przyszło
pokrycie = trafienia / (trafienia + przegapione)    ile z tego, co przyszło, obiecaliśmy
```

Wynik backtestu wraca do systemu jako **kalibracja pewności** tego klienta (wygładzona, ograniczona do przedziału 0,60–1,15) — to jedyne miejsce, w którym prognoza sama się ostrzy w miarę napływu zamówień. Kolejność jest krytyczna: backtest biegnie na surowej historii i dopiero jego wynik wchodzi do prognozy; odwrotnie oznaczałoby sprzężenie zwrotne mnożnika na własne wejście.

Zweryfikowane na danych: klient szpitalny z 60 zamówieniami — 7 predykcji, 90% odtworzonej trafności.

11 Pulpit dystrybutora — co z tym zrobić rano

Wycena i prognoza są bezużyteczne, dopóki ktoś nie wie, od czego zacząć dzień. Pulpit szereguje sygnały po tym, czego *nie da się cofnąć*.

„Co dziś zrobić” — pięć rodzajów sygnału

SYGNAŁ	WAGA	DLACZEGO TAKA WAGA
Towar traci termin (30 dni)	100	Po terminie to odpis, którego nic nie cofnie
Zapytania bez odpowiedzi	85	Nie ma dokumentu do ponownego wystawienia — klient widzi wyłącznie ciszę
Oferty bez odzewu (3 dni)	70	Ofertę można wystawić ponownie; kosztuje deal, nie towar
Zamówienia do skompletowania	50	Przychód już zaksięgowany, spóźnienie kosztuje poziom obsługi
Zapas poniżej bezpieczeństwa	40	Przed nami lead time dostawcy, ale żadnego jednodniowego urwiska

```
presja_terminu = 0 (brak terminu lub >14 dni) · 14 - dni_do_terminu · 14 (po terminie)
wynik = waga_nieodwracalności × (14 + presja_terminu)
```

Presja może co najwyżej **podwoić** wagę, więc bliski termin podnosi lżejszy sygnał ponad cięższy, który jest jeszcze dwa tygodnie przed sobą — ale przy porównywalnych terminach nieodwracalność zawsze wygrywa. Widget bez niczego pilnego pokazuje pusty stan, a nie pięć zer.

Pozostałe widżety pulpitu

Towar traci termin Dwa liczniki (wygasa wkrótce / po terminie) i do 10 partii posortowanych po dacie ważności, z dostępną ilością i linkiem do partii. Punktem odniesienia dat jest znacznik z danych, nie zegar przeglądarki.

Braki magazynowe	Pozycje (wariant × magazyn) na progu zamówienia lub poniżej, oraz ile z nich jest już pod zapasem bezpieczeństwa. Zagnieżdżenie tych zbiorów jest sprawdzane przy każdym odczycie — nic w danych nie wymusza, żeby zapas bezpieczeństwa był niższy od progu zamówienia.
Przewidywane zamówienia	Zakładka na karcie klienta: rytm zamawiania, trafność historyczna, do 8 kart spodziewanych dostaw z rozwijanymi liniami i dowodem („co 9 dni”, „najczęściej we wtorek 16/22”, „ostatnio ...”), sekcja „Pominięto” i notatki operatora. Dwie akcje: „Policz cenę” (razem kontra osobno) i „Utwórz szkic zamówienia”.
Marża na dokumencie	Zakładka na zamówieniu i ofercie: zysk, marża, narzut, wodospad kosztów per pozycja i panel „co ta cena zakłada”.

Ekran „Przewidywane zamówienia” — przekrój przez wszystkich klientów

Horyzont 3 / 7 / 14 / 30 dni, dwa widoki (dostawy albo produkty), filtr dni tygodnia z licznikami, dwie sekcje — „**Nie przyszły — warto zadzwonić**” i „**Nadchodzące**” — oraz eksport CSV. W widoku produktowym te same linie są przegrupowane po produkcie: łączna ilość, liczba klientów, ilu się spóźnia, pewność jako średnia ważona ilością.

Porównanie cen — jak wyceniał ERP, jak wycenia silnik

Jeden zaimportowany dokument wyceniony dwa razy: tak, jak zaksięgował go system źródłowy, i tak, jak liczy go silnik cost-to-serve. Obie strony liczą **ten sam koszt towaru** — silnik bierze go wprost z pozycji dokumentu (rozdział 03), więc jedyne, co je różni, to koszt obsługi; inaczej porównanie pokazywałoby różnicę metody wyceny jako różnicę zysku. Ekran zestawia obie struktury kosztu, realną trasę dowozu (kilometry, przystanki, paliwo, kierowca, koszt stały auta), drabinę wolumenu $\times 1 / \times 2 / \times 3 / \times 5 / \times 10$ i — dla każdej pozycji — cenę możliwą do zaproponowania:

```
cena_oferty = koszt_towaru + koszt_obsługi_przy_tym_zachowaniu + realny_zysk_jednostkowy_z_dokumentu
```

Czyli: klient dostaje dokładnie tyle, ile jego zachowanie oszczędza w koszcie obsługi, a zysk dystrybutora w złotych zostaje taki, jaki był. To jest liczba, którą można pokazać klientowi.

12 Portal klienta — powierzchnia zamówieniowa, nie kalkulator

Klient przegląda katalog, składa koszyk i widzi cenę, którą zapłaci. Nic więcej.

Silnik zwraca komplet: koszt zakupu, koszt obsługi, narzut, marżę, jedenastokładnikowe rozbiecie ze stawkami pracy, identyfikator kalkulacji i ostrzeżenia. Do portalu przechodzi wyłącznie { produkt, SKU, ilość, cena jednostkowa netto, wartość netto } — budowane **pole po polu z jawnej listy dozwolonych**, nigdy przez „skopiuuj i usuń wrażliwe”, bo wtedy każde nowe pole silnika wyciekałoby domyślnie. Test serializuje odpowiedź i przeszukuje ją pod kątem zakazanych ciągów, więc ta gwarancja przeżywa refaktor typów.

SKALA PROBLEMU

Zaseedowana linia wycenia się na 83,52 PLN przy koszcie zakupu 26,07 PLN — i obie liczby są w jednym obiekcie wyniku.

Koszyk przysyłany przez przeglądarkę to wyłącznie pary produkt + ilość. Klient, grupa, strefa dostawy, scenariusz i waluta są *celowo nieobecne* — silnik wyprowadza je z profilu klienta, więc kupujący nie może „pochodzić po tańszej strefie dostawy” ani wycenić się jako inna firma. Przy składaniu zgłoszenia ceny są **przeliczone po stronie serwera**; nic, co przeglądarka powiedziała o pieniądzach, nie jest używane. Wyceny portalowe nie trafiają do księgi audytowej, bo nie stoi za nimi żaden użytkownik operatora.

13 Wejście do sprzedaży – tryb obserwacyjny

Silnik nie zmienia dziś ani jednej kwoty na fakturze. Zamiast tego mierzy rozjazd między tym, co faktycznie zafakturowano, a tym, czego chciałby silnik – zanim ktokolwiek zdecyduje o przełączeniu go w tryb decyzyjny.

Mechanizm to **dekorator** nałożony na usługę wyliczającą sumy dokumentu sprzedaży. Przy każdym przeliczeniu sum wycenia ten sam koszyk i zapisuje różnicę – *obiekt wyniku dekorowanego wywołania wraca przez referencję, nietknięty*.

PYTANIE	ODPOWIEDŹ W KODZIE
Co jest zapisywane	cena zafakturowana, cena silnika, różnica bezwzględna i procentowa ze znakiem – dodatni znaczy, że silnik chce więcej, niż zafakturowano. Kierunek jest całym sensem tego ekranu.
Czy to spowolni sprzedaż	Wyłączone domyślnie flagą środowiskową. Sumy dokumentu przeliczają się przy niemal każdej zmianie szkicu, a jedno przejście to profil dostawcy, cały zestaw parametrów, snapshot katalogu i magazynu plus jedenastokrotny potok.
Czy zaśmieci księgę	Deduplikacja po tożsamości pozycja + cena faktury + cena silnika. Sumy przeliczają się z sześćdziesięciu miejsc w module sprzedaży – bez tego powstawałby jeden wiersz na pozycję na każde przerysowanie. Druga obserwacja powstaje tylko wtedy, gdy zmieniło się samo porównanie.
Czy może zepsuć zamówienie	Zapis idzie przez odrębną jednostkę pracy, poza transakcją sprzedaży, a każda awaria jest tykana z ostrzeżeniem w logu: <i>obserwacja, która psuje sprzedaż, jest gorsza niż brak obserwacji</i> .
Różne waluty	Porównanie jest pomijane, nie zapisywane. Wiersz nie ma kolumny mogącej oznaczyć porównanie jako nieważne, a różnica między dwiema walutami czytaby się jak prawdziwy rozjazd.

CO „TRYB SHADOW” ZNACZY DOKŁADNIE, A CZEGO NIE ZNACZY

mode jest **etykietą raportowaną**, nie zabezpieczeniem. Nic w potoku go nie czyta – jest stemplowany na wyniku i na wierszu księgi, żeby audyt wiedział, co operator sądził, że silnik wtedy robi. Bezpieczeństwo bierze się stąd, że *żaden kod w tym pakiecie nie zapisuje kwoty sprzedaży*, a nie z wartości tego pola.

Zaplanowana kolejność wchodzenia dalej: (1) obserwacje bez mutacji – **zrobione**; (2) raportowanie trybu i UI tylko do odczytu; (3) sterowanie trybem z uprawnieniami i blokadą optymistyczną; (4) rozszerzenie kontekstu sprzedaży o identyfikator dokumentu i klienta – dotyka rdzenia platformy; (5) tryb decyzyjny, po obejrzeniu danych z obserwacji. Kroki 0–3 zostawiają wyjście sprzedaży bit w bit identyczne.

14 Powierzchnia rozwiązania

Ekran silnika wyceny

kalkulator	Pulpit wyceny – koszyk, marża, cel i podłoga, propozycje doradcy wprowadzane jednym kliknięciem. Stan koszyka siedzi w adresie strony, więc odświeżenie lub wystany link otwiera dokładnie tę samą wycenę
playground	Wycena koszyka z pełnym wodospadem
advisor	Propozycje upsellowe z obiema stronami bilansu

Ekran pulpitu dystrybutora

predicted-orders	Przewidywane zamówienia wszystkich klientów
pricing/comparison	Porównanie wyceny ERP i silnika
portal/ordering	Portal zamawiania dla klienta

Wstrzyknięcia w cudze ekran

dokument	Zakładka „Marża” na zamówieniu i ofercie
----------	--

deadstock	Rotacja i zapas martwy z rekomendowaną podłogą	klient	Zakładka „Przewidywane zamówienia” i zakładka cennika
coverage	Rejestr pokrycia danych	pulpit	Trzy widgety dashboardu
shadow	Obserwacje rozjazdu cen	menu	Pulpit jako pierwsza pozycja grupy „Sprzedaż”
params × 11	Stawki pracy, kroki procesu, scenariusze, opakowania, magazyn, pojazdy, paliwo, strefy dostaw, pozycje zakupowe, reguły marżowe, bezpieczniki, cele i wagi — lista, dodawanie, edycja		

PULPIT WYCENY JAKO PUNKT ZBIEGU

Sześć innych powierzchni prowadzi do niego z gotowym koszykiem: przewidywane zamówienia, prognoza na karcie klienta, porównanie cen, doradca, zakładka marży na dokumencie i zakładka cennika klienta. Wszystkie budują adres przez jedną funkcję pomocniczą, nigdy ręcznie — dzięki temu „policz to porządnie” jest jednym kliknięciem z każdego miejsca, w którym operator zobaczy powód, żeby to zrobić, a kontrakt adresu ma jedno źródło prawdy objęte testami.

API

ENDPOINT	UPRAWNIENIE	PRZEZNACZENIE
POST /api/pricing/quote	pricing.quote	Wycena koszyka, zapis do księgi
POST /api/pricing/simulate	pricing.simulate	Wariant „co gdyby” — nie zapisuje niczego
POST /api/pricing/advice	pricing.simulate	Propozycje, krzywa wolumenu, podłogi
GET /api/pricing/calculations/:id	pricing.audit.read	Odtworzenie zapisanej wyceny z rozbiciem
GET /api/pricing/coverage	pricing.audit.read	Rejestr pokrycia danych
GET /api/pricing/deadstock*	pricing.view	Lista, podsumowanie; decyzje pod osobnym uprawnieniem
CRUD /api/pricing/<parametr>	pricing.params.*	11 rodzin parametrów
GET .../order-forecast	forecast.view	Prognoza klienta (JSON / CSV / XML / Markdown)
GET .../order-forecast/upcoming	forecast.view	Przekrój przez wszystkich klientów
POST .../basket-pricing	pricing.compare	Koszyk razem kontra osobno
POST .../portal/quote requests	sesja klienta	Wycena i złożenie zgłoszenia z portalu

Łącznie 23 trasy w silniku i 9 w module pulpitu. Wszystkie walidowane schematami Zod na wejściu i wyjściu, wszystkie z opisem OpenAPI, zakres dzierżawcy i organizacji zawsze z kontekstu uwierzytelnienia — nigdy z ciała żądania.

Uprawnienia

Dziesięć uprawnień silnika (pricing.view, quote, simulate, params.read/write, audit.read, deadstock.summary/decide, supplier.import, mode.change) i sześć uprawnień pulpitu. Uprawnienia pulpitu **same z siebie nie otwierają żadnych danych** — każda bramka wymaga ich *obok* uprawnień modułu właścicielskiego. Gotowa rola distributor to 60 uprawnień z domknietymi zależnościami; świadomie *nie* zawiera zmiany trybu silnika, importu dostawcy, edycji numeracji dokumentów ani administracji użytkownikami.

Reszta powierzchni

21 encji	Parametry dystrybutora, profile klientów, bezpieczniki, rejestr pokrycia, księga kalkulacji (dopisywalna, bez edycji), obserwacje trybu cienia, decyzje o zapasie martwym
11 komend CLI	Seed i czyszczenie danych demonstracyjnych, 9 seederów danych referencyjnych — wszystkie idempotentne, wszystkie z <code>--dry-run</code>
5 języków	1144 klucze tłumaczeń (en, pl, es, de, ko) z testem wymuszającym identyczne zestawy kluczy — żaden komunikat użytkownika nie jest zaszyty w kodzie
blokada optymistyczna	Każda edytowalna encja parametru niesie wersję; równoległa edycja kończy się czytelnym konfliktem, nie cichym nadpisaniem

15 Dane referencyjne — dystrybutor HoReCa

Żeby cokolwiek z tego dało się pokazać, potrzebny był realistyczny zbiór danych. Powstał jako zestaw idempotentnych seederów, oznaczony flagą danych demonstracyjnych i usuwalny jedną komendą.

katalog	200 produktów w PLN, 22 kategorie, 87 wariantów opcyjnych, 165 domyślnych, 400 przeliczników jednostek	magazyn	1 magazyn, 48 lokalizacji, 200 stanów (16 bez towaru, 26 pod progim), 5520 ruchów
zakupy	204 pozycje zakupowe z ceną z ostatniej dostawy i progami rabatowymi	partie	92 partie z terminami; 14 traci termin w 30 dni — 20 227 PLN ryzyka odpisu
klienci	40 firm, 48 adresów z rzeczywistymi współrzędnymi, 40 profili cenowych	reguły marżowe	19 grup produktowych, 5 segmentów klientów, 2 ustalenia indywidualne, 3 limity
strefy	4 strefy dostaw <i>wyliczone</i> ze współrzędnych klientów, ze współczynnikiem krętości drogi 1,35	konta	1 operator dystrybutora, 6 kont portalowych klientów

Stawki startowe są **załoženiami, nie pomiarami**, i tak są oznaczone: robocizna 48–90 PLN/h z narzutem 22%, miejsce paletowe 95 PLN/miesiąc, koszt kapitału 9% rocznie, rotacja 30 dni, van 3,5 t przy 11,5 l/100 km i 4200 PLN kosztu stałego miesięcznie, diesel 6,49 PLN/l, narzut docelowy 66% (mediana z ksiąg dystrybutora), marża minimalna 8%, limit rabatu 25%.

16 Stan, weryfikacja i co pozostaje otwarte

Zweryfikowane

908 testów	619 w silniku wyceny, 289 w module pulpitu — wszystkie przechodzą
przypadki złote	Cena referencyjna przypięta co do grosza; zmiana wymaga świadomej edycji testu
niezmienniki	Wodospad sumuje się do ceny; cena $\times$ ilość = wartość pozycji; ta sama data daje tę samą cenę

Świadomie niezrobione

- Dwa składniki potoku — profil klienta (brak danych wejściowych) i efekt wolumenu (zastąpiony amortyzacją kosztów stałych)
- Punkt nadania firmy i wyliczanie odległości ze współrzędnych — strefy trzeba nadal przypisywać
- Przebudowa nawigacji pod personę dystrybutora
- Tryb decyzyjny w sprzedaży — kroki 4–5, wymagają zmiany w rdzeniu platformy
- Wysyłanie przewidywanego koszyka klientowi do akceptacji — potrzebuje własnego rekordu propozycji,

Zweryfikowane uruchomieniem obu zestawów w dniu
sporządzenia dokumentu, po zmianie wprowadzającej drugie
źródło kosztu zakupu.

Pytania, na które potrzebna jest decyzja biznesowa

1. **Realna wartość limitu rabatu** — dziś 25% z danych demonstracyjnych. Bezpieczniki już umożliwiają różne wartości per grupa klientów.
2. **Czy brak ceny zakupu ma blokować wycenę** — dziś świadomie zwraca zero z ostrzeżeniem, zamiast odmawiać.
3. **Czy korekta gabarytowa ma mnożyć cały koszt**, czy tylko magazyn i dostawę — dziś mnoży całość.
4. **Realna stawka utylizacji** dla drabinki terminowej — dziś 2,50 PLN/kg (6,00 dla chemii niebezpiecznej) jako założenie.
5. **Kiedy włączyć obserwacje trybu cienia na produkcji** — mechanizm jest gotowy i wyłączony; dopiero jego dane uzasadnią jakąkolwiek rozmowę o trybie decyzyjnym.

PODSUMOWANIE W JEDNYM ZDANIU

Rozwiązanie zamienia cenę z wyniku negocjacji w **wynik obliczenia, które da się przeczytać na głos** — z pełnym kosztem obsługi konkretnego klienta, z bezpiecznikami, których nie da się obejść przypadkiem, z jawną listą tego, czego system nie wie, i bez dotykania ani jednej kwoty na fakturze, dopóki ktoś świadomie tego nie zdecyduje.

Open Mercato · @open-mercato/pricing-engine + distributor_workspace · stan na 19 września 2026. Dokument opisuje kod znajdujący się w repozytorium; liczby pochodzą z testów, seederów i pomiarów na zbiorze referencyjnym.